

DATA STRUCTURE AND ALGORITHM

DATA STRUCTURE AND ALGORITHM

Ins

(1)

(2)

(3)

(4)

Page No. 19
Date: / /

Instructor's Note

- (1) Finish atleast 2 hours per day, so that finish it in month
- (2) At the end of video, you can start coding by yourself.
- (3) While watching analysis video, practice it by yourself. It will help in remembering analysis & applying them at right place
- (4) For teaching analysis, notation hasn't been given that is present in last

Revise using pen & paper.

Teacher's Note

- (1) Finish atleast 2 hours per day so that finish it in time
- (2) At the end of class you can start taking 1 hour
- (3) While watching analysis video practice it by yourself. It will help in understanding analysis & applying them at right place
- (4) For teaching analysis relation doesn't depend on that is present in text

20/12/20
Dec. 2020
20/12/20

Introduction

Page No. 21

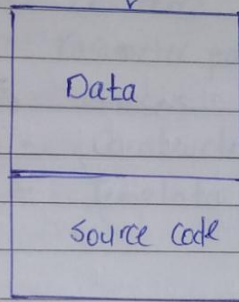
Date: / /

Languages

- C
- C++
- Java
- C#
- Python
- JavaScript
- ...

STL
collection

C
P
U



having built-in
data structure

Topics

Physical data
- structure

- Arrays
- Matrices
- Linked List

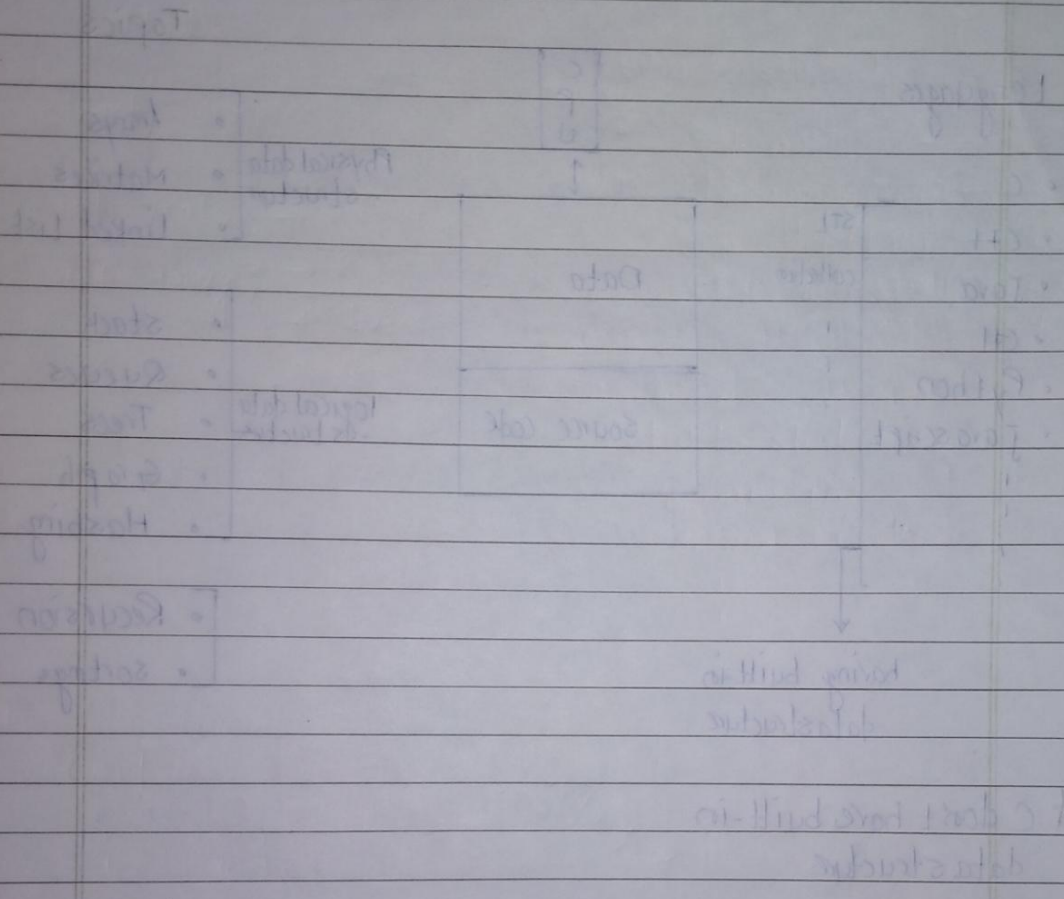
logical data
- structure

- Stack
- Queues
- Trees
- Graph
- Hashing

- Recursion
- Sortings

* C don't have built-in
data structure

Introduction



Essential C and C++ concepts

Page No. 28

Date: / /

1. Arrays
2. structure
3. pointers
4. Reference
5. Parameter passing
6. classes
7. Constructor
8. Templates

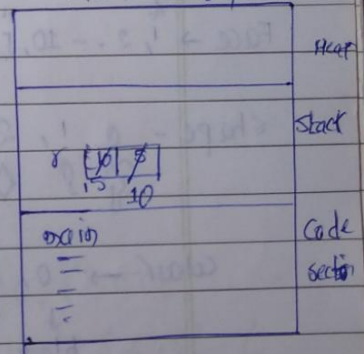
Arrays



Structure

1. Defining structure
size of structure
Declaring a structure
Accessing members

Main memory



struct Rectangle

int length; → 2 bytes
int breadth; → 2 bytes
+ 1 bytes

int main()

{
 struct Rectangle r;

 Declⁿ + initⁿ struct Rectangle r = {10, 5};
 r.length = 15; r.breadth = 10;

 Printf("Area of Rectangle is %d", r.length * r.breadth);

}

1. Complex No:
atrib

```
struct complex
{
    int real; → 2 bytes
    int imag; → 2
                4 bytes
}
```

2. Student

```
struct student
{
    int roll; — 2
    char name[25]; — 25
    char dept[10]; — 10
    char address[50]; — 50
                        77 bytes
}
```

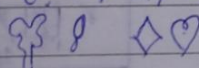
```
int main()
{
    struct student s;
```

```
    s.roll = 10;
    s.name = "John";
```

```
    ;
}
```

3. Card

Face → 1, 2, ..., 10, J, Q, K

shape - 0, 1, 2, 3


colour → 0, 1
 ↓ ↓
 black Red

struct Card

```
{
    int face; — 2
    int shape; — 2
    int colour; — 2
                    6 bytes
}
```



```

int main()
{
    struct card c;
    c.Face = 1;
    c.shape = 0;
    c.colour = 0;
    or
    struct card c = {1, 0, 0};
}

```

	c
face →	1
shape →	0
colour →	0

IF you want deck of card
 ↓
 it can be done with the help of array

```

struct card
{
    int face;
    int shape;
    int color;
};

int main()
{
    struct card deck[52] = {1, 0, 0, 2, 0, 0, ...
    1, 1, 0, 0, 2, 1, 0, 0};
}

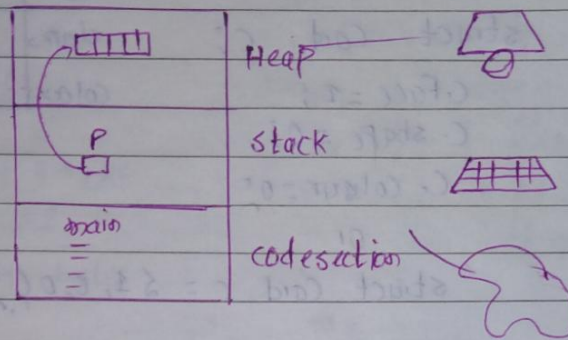
```

```
printf("r.d", deck[0].face);
```

```
printf("r.d", deck[0].shape);
```

```
⋮
```

Pointers



Uses

- ① accessing Heap
- ② accessing Resources
- ③ Parameter passing

For allocating memory in heap using C

```
#include <stdlib.h>
```

```
int main()
```

```
{
    int *P;
```

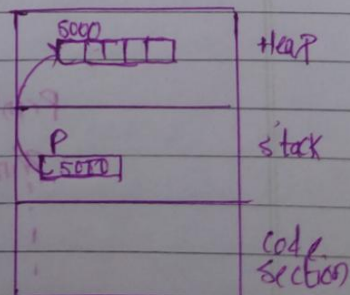
* malloc returns
void pointer

```
C → P = (int *) malloc (5 * sizeof(int));
```

```
C++ → P = new int[5];
```

```
C → Free (P);
```

```
C++ → delete [] P;
```



alias
or
Reference (only in c++) → not inc

Page No. 27

Date: / /

```
int main()
{
    int a = 10;
    int &r = a;

    cout << a; → 10
    r++;
    cout << r; → 11
    cout << a; → 11
}
```

alias
a/r
10/11
200/201

Pointer to a structure

```
struct Rectangle
{
    int length; - 2
    int breadth; - 2
};
```

4 bytes

```
int main()
{
    struct Rectangle r = {10, 5};
    struct Rectangle *P = &r;

    r.length = 15;
}
```

✓ (*P).length = 20;
OK

✓ P → length = 20;

*P.length = 20 X

↓
• operator is given precedence so it is like

↓
*(P.length)

Allocating in heap

```

struct Rectangle
{
    int length;
    int breadth;
};
  
```

```

int main()
{
    struct Rectangle *P;
  
```

```

    P = (struct Rectangle *) malloc(sizeof(struct Rectangle));
  
```

P → length = 10;

P → breadth = 5;

Let the Compiler
decide the size

Functions

```

int add (int a, int b)
  
```

↓
Formal Parameter

```

int main()
  
```

```

{
    x = add(x, y)
  
```

→ Actual Parameter

Parameter passing method

Array as Parameter

int *A
or
void fun(int A[], int n)

{
int i;

for (i=0; i<n; i++)

printf("%d", A[i]);

{
int main()

{ int A[5] = {2, 4, 6, 8, 10};

fun(A, 5);

}

Returning Array

int * fun(int n)

{ int *P;

P = (int *) malloc (n * sizeof(int));

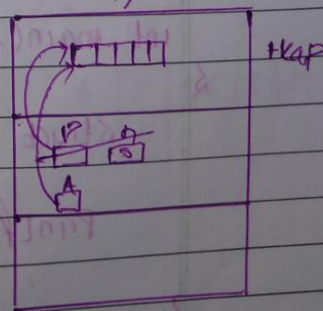
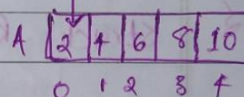
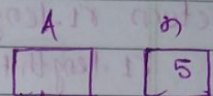
return (P);

int main()

{ int *A;

A = fun(5);

* Understand it as pointer, the only difference is that it is specially for array & can be used for other also.



Structure as Parameter

call by value

Formal Parameter

struct Rectangle

int area(struct Rectangle r1)

int length;
int breadth;

return r1.length * r1.breadth;

r1.length++;

int main()

struct Rectangle r = {10, 5};

printf("area: %d", area(r));

actual parameter

↑

length	10
breadth	5

* A new object will be created & everything will be copied

length	10
breadth	5

Call by reference

int area(struct Rectangle *r1)

return r1->length * r1->breadth;
r1->length++;

int main()

struct Rectangle r = {10, 5};

printf("area: %d", area(&r));

↑

length	10
breadth	5

* In call by reference, a new object will not be created

Call by address

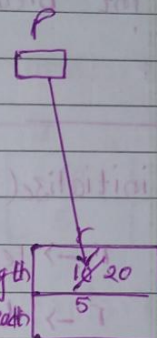
→ Program For changing length

struct Rectangle
{ int length;
int breadth;

void changelength(struct Rectangle *P, int l) {

{
P->length = l;

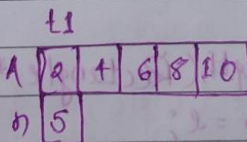
}
int main() {
struct Rectangle r = {10, 5};
changelength(&r, 20);



see

void fun(struct Test t1) {
t1.a[0] = 10;

struct Test
{ int a[5];
int n;

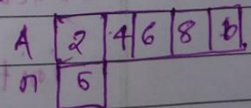


→ A new memory created & everything got copied

int main() {

struct Test t = {2, 4, 6, 8, 10, 5};

fun(t);



Structures and Functions

struct Rectangle

{

int length;

int breadth;

};

void initialize(struct Rectangle *r, int l, int b)

{

r->length = l;

r->breadth = b;

}

int area(struct Rectangle r)

{

return r.length * r.breadth;

}

void changeLength(struct Rectangle *r, int l)

{

r->length = l;

}

int main()

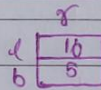
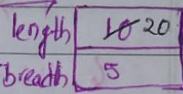
{ struct Rectangle r;

initialize(&r, 10, 5);

area(r);

changeLength(&r, 20);

}



Converting a program to a C++ class

class Rectangle

{ private:

int length;

int breadth;

public:

Rectangle (int a, int b)

{ length = a;

breadth = b;

{

int area()

{ return length * breadth;

{

void changelength(int a)

{ length = a;

{

};

int main()

{ Rectangle r (10, 5);

r.area();

r.changelength(20);

{

	r
length	10
breadth	5

C++ class and constructor

```
#include <iostream>
using namespace std;

class Rectangle
{
private:
    int length;
    int breadth;
public:
    Rectangle() { length = breadth = 1; }
    Rectangle(int a, int b);
    int area();
    int perimeter();
    int getLength() { return length; }
    void setLength(int a) { length = a; }
    ~Rectangle();
};

int main()
{
    Rectangle r(10, 5);
    cout << r.area();
    cout << r.perimeter();

    r.setLength(20);
    cout << r.getLength();
}
```


Template classes

```
int Arithmetic::add()
{
    int c;
    c = a + b;
    return c;
}
```

```
int Arithmetic::sub()
{
    int c;
    c = a - b;
    return c;
}
```

Class Arithmetic

{ Private:

int a;

int b;

Public:

Arithmetic(int a, int b);

int add();

int sub();

{

Arithmetic::Arithmetic(int a, int b)

{ this->a = a;

this->b = b;

}

data type Above is working for int data type, for writing to float, we have to create another class so we write generic class ↓**

* OF which data type you want to create object

```
int main()
{
    Arithmetic<int> ar(10, 5);
    cout << ar.add();
    Arithmetic<float> ar1(10, 12);
    cout << ar1.add();
}
```

template <class T>

Arithmetic<T>::add()

{

T c;

c = a + b;

return c;

}

template <class T>

Arithmetic<T>::sub()

{

T c;

c = a - b;

return c;

}

template <class T>

Class Arithmetic

{ Private:

T a;

T b;

Public:

Arithmetic(T a, T b);

T add();

T sub();

→ * effect of above template class ends here

template <class T>

Arithmetic<T>::Arithmetic(T a, T b)

{ this->a = a;

this->b = b;

}

template <class>

class Arithmetic

{ private:

int a;

int b;

public:

Arithmetic(int a, int b)

{ int add();

int sub();

};

Arithmetic::Arithmetic(int a, int b)

{ this->a = a;

this->b = b;

}

↓
data type
template to create the class
No right generic class

template <class T>

class Arithmetic

{ private:

T a;

T b;

public:

Arithmetic(T a, T b)

{ add();

sub();

};

template <class T>

Arithmetic::Arithmetic(T a, T b)

{ this->a = a;

this->b = b;

}

int Arithmetic::add()

{ int c;

c = a + b;

return c;

}

int Arithmetic::sub()

{ int c;

c = a - b;

return c;

}

template <class T>
+ T a, b;
to create object
T Arithmetic::add(T a, T b)

{ int result;

};

T c;

c = a + b;

return c;

conf << a + b;

Arithmetic::add(T a, T b)

{ Arithmetic::Arithmetic(T a, T b)

conf << a + b;

return c;

}

Page No. 36

Date : / /

Page No. 37

Date : / /

Page No. 38
Date: / /

Page No. 38
Date: / /

1.

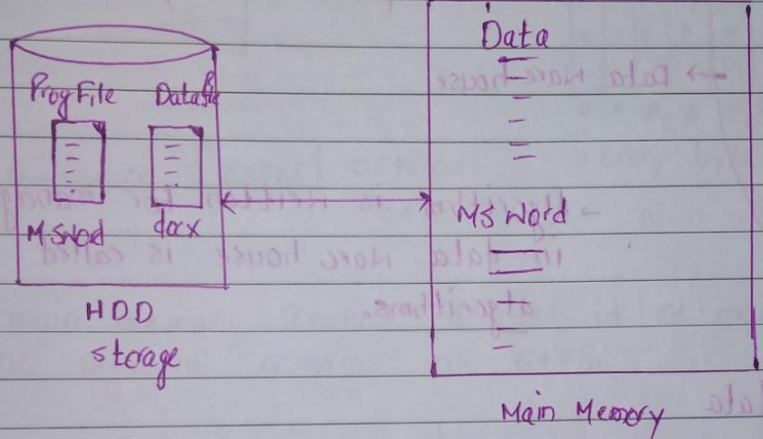
Data structure

Page No. 39

Date: / /

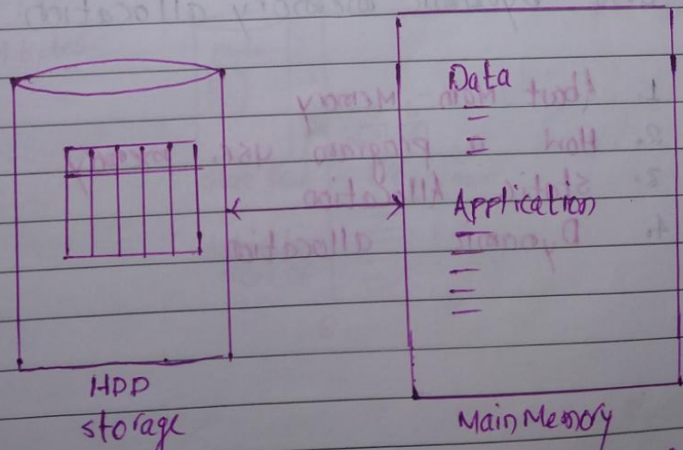
1. Data structure → It is arrangement of data ^{in Main memory} so that it can be used efficiently
2. Data base
3. Data Warehouse
4. Big Data

1. Data structure



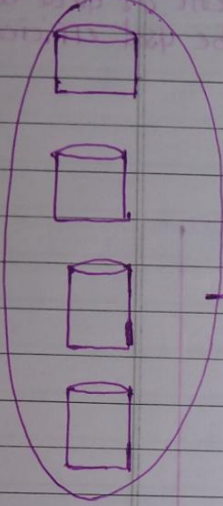
2. Data base

When data is very big or there is commercial data then there is use of data base



Data base → Arranging the data in some model like say relational model in the permanent storage so that it can be retrieved or accessed by appⁿ easily. Arrangement is called database

3. Data Ware house



→ Data Ware house

historical data [inactive data]

or

→ 10 years of data or 50 years of data kept in array of disc

not operational means not used day today

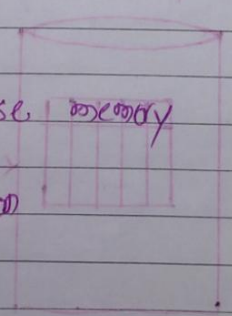
→ Algorithm is written for managing data in data ware house is called Data mining algorithms.

4. Big data

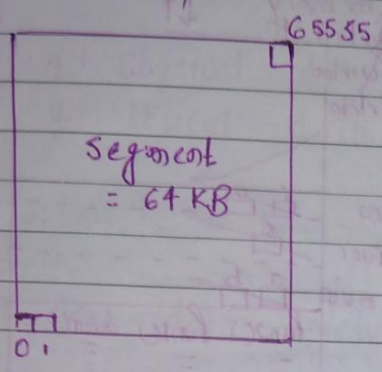
↓ storing & analysing that big size data, that study is called Big data.

Static and Dynamic memory allocation

1. About Main Memory
2. How a program use memory
3. static Allocation
4. Dynamic allocation



1. Main Memory

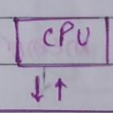


$0 - 65535 = 65536 = 64 \times 1024 = 64 \text{ KB}$

Every byte has address

Our main memory remains divided in so many segments & one segment remains of 64KB.

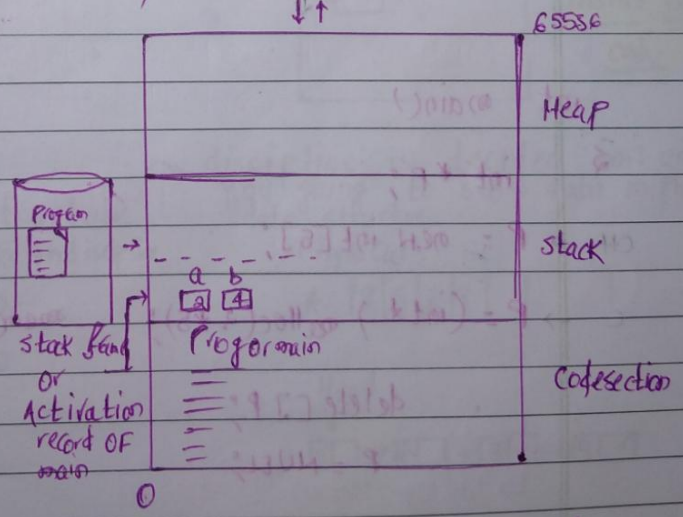
2. How Program uses memory



void main()

int a; - 2 bytes

float b; - 4 bytes



```
void Fun2(int i)
```

```
{ int a;
```

```
{
```

```
void Fun1()
```

```
{ int x;
```

```
Fun2(x);
```

```
{
```

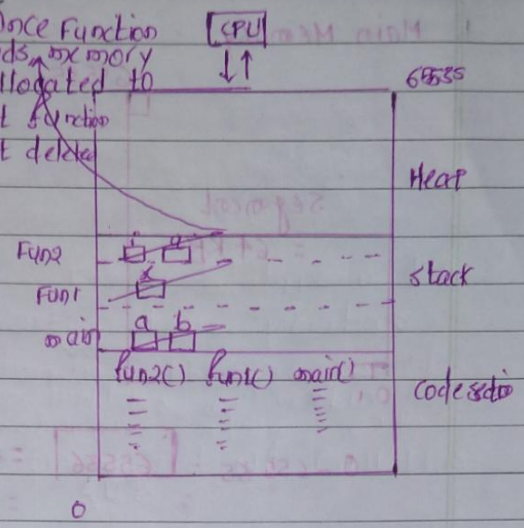
```
void main()
```

```
{ int a;
```

```
float b;
```

```
{ Fun1();
```

Once Function ends, memory allocated to that function get deleted



* Used for unorganised thing

how heap memory is utilised

```
int main()
```

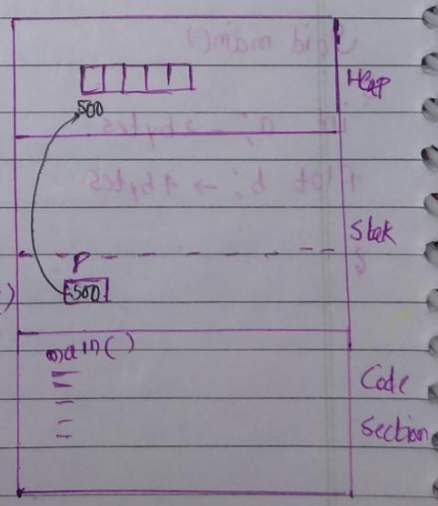
```
{ int *P;
```

```
CH -> P = new int[5];
```

```
C -> P = (int*) malloc(3*5);
```

```
delete [] P;
```

```
P = NULL;
```

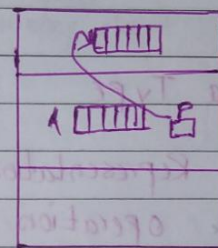
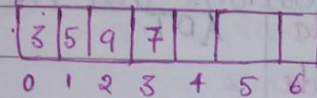


Types of Data structures

1. Physical data structure
2. Logical data structure

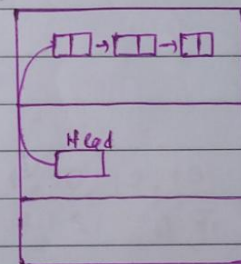
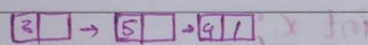
1. Physical data structure

Array



Array can be
created inside
heap & stack

Linked List



Linked List
Created
inside heap
only

2. Logical data structure

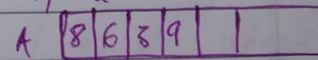
Logical

Notes and discipline

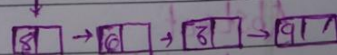
- | | | |
|------------|---------------|------|
| Linear | 1. Stack | LIFO |
| | 2. Queues | FIFO |
| non-linear | 3. Trees | |
| | 4. Graph | |
| Table | 5. Hash Table | |

discipline is decided how are
you going to store data in physical
data structure

Physical



Head



Using logical data structure we do implementation on Array, linked List or combination of array & linked List

Abstract Data type

1. What is ADT
2. List ADT

1. What is ADT $\rightarrow$ * user defined datatype is defined along with their operations

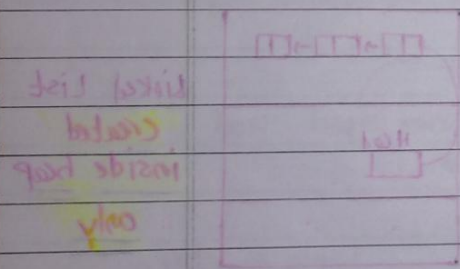
Data Type

To simplify the process of solving problems, we combine the data structure along with operations and we call this

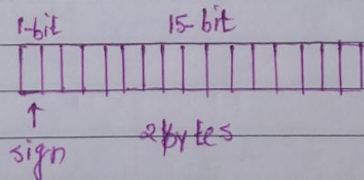
1. Representation of Data
2. operation on Data

ADT.

1. Representation of Data



int x;



2. Operation on Data

Abstract
 $\rightarrow$ hiding internal detail

in memory, work is going on binary, we can do our work without knowing it that is Abstract Form

List $\rightarrow$ 8, 3, 9, 4, 6, 10, 12
 0 1 2 3 4 5 6

Data representation:-
 it is abstract Forme

1. Space For storing element
2. Capacity
3. Size $\rightarrow$ Max capacity
 $\rightarrow$ how many element

1. Array
 2. Linked List

Operations:-

add(x)

remove()

search(Key)

List $\rightarrow$ 8, 3, 9, 4, 6, 10, 12, 15
 0 1 2 3 4 5 6 7

- add(element) / append(ele)
- add(index, ele) / insert(index, ele)
- remove(index)
- set(index, ele) / Replace(index, ele)
- get(index)
- search(Key) / contains(Key)
- sort()

Time and Space Complexity

depends on
no of element

Page No. ⁺⁶
Date: $O(n) \rightarrow$ space complexity

Time Complexity

For ($i=0; i < n; i++$)

Space complexity
1 2 3 4 5 6 7 8 9 $\rightarrow O(n)$

$O(n) \rightarrow$ order or degree of n

first ex $\rightarrow$ adding

1 2 3 4 5 6 7 8 9 $\rightarrow O(n)$

For ($i=0; i < n; i++$)

$n \times n = n^2$

For ($j=0; j < n; j++$)

$O(n^2)$

ex $\rightarrow$ For each element going to each element

For ($i=0; i < n; i++$)

1 + 2 + 3 + 4 + ... + (n-3) + (n-2) + (n-1)

$= \frac{n(n-1)}{2} = \frac{n^2 - n}{2} = \frac{n^2}{2} - \frac{n}{2}$

For ($j=i+1; j < n; j++$)

$O(n^2)$

ex $\rightarrow$ For each element, we are going to next remaining element

Array of linked list

1	2	5	9	6	4	12	15	8	5	7
0	1	2	3	4	5	6	7	8	9	

each time divided by 2

For($i=n$, $i>1$, $i=i/2$)

OR
 $i=n$
While($i>1$)

$\log n$
 $\log_2 n$
as divided by 2

$i=i/2$

For($i=0$; $i<n$; $i++$)

For($j=0$; $j<n$; $j++$)

	0	1	2	3
0	8	3	5	9
1	7	6	4	2
2	6	5	3	9
3	10	4	6	2

$O(n^2)$

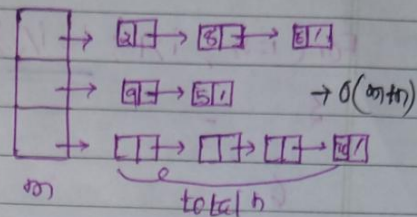
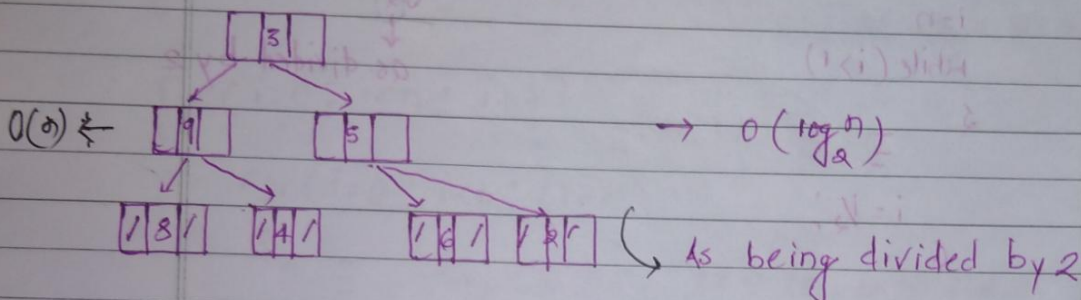
$O(n^2)$

processing each element of matrix

$O(n)$

processing only one row

Array of Linked list

 $\rightarrow O(m+n)$  $O(n) \leftarrow$ $\rightarrow O(\log n)$

Time & Space Complexity From Code

Every statement in program take one unit of time in execution

ex: Arithmetic operation

assignment

or

conditional statement

For complex statement, it needs to be studied in detail

void swap(x, y)

{ int t;

t = x; — 1

x = y; — 1

y = t; — 1

{

 $f(n) = 3n$ $O(n) = O(1)$


```
int sum (int A[], int n)
```

```
{
    int s, i;
```

```
    s = 0;
```

```
    For (i = 0; i < n; i++)
```

```
{
```

```
        s = s + A[i];
```

```
    }
```

```
    return s;
```

```
}
```

$$f(n) = 2n + 3$$

$$\downarrow$$

$$O(n)$$

```
void add (int n)
```

```
{
    int i, j;
```

```
    For (i = 0; i < n; i++)
```

```
{
```

```
        For (j = 0; j < n; j++)
```

```
{
```

```
            C[i][j] = A[i][j] + B[i][j];
```

```
        }
```

```
    }
```

$$f(n) = 2n^2 + 2n + 1$$

$$\rightarrow O(n^2)$$

* Complex statement

```

FUN1()
{
    FUN2();
}

FUN2()
{
    for(i=1; i<n; i++)
    {
    }
}
  
```

→ $O(n)$

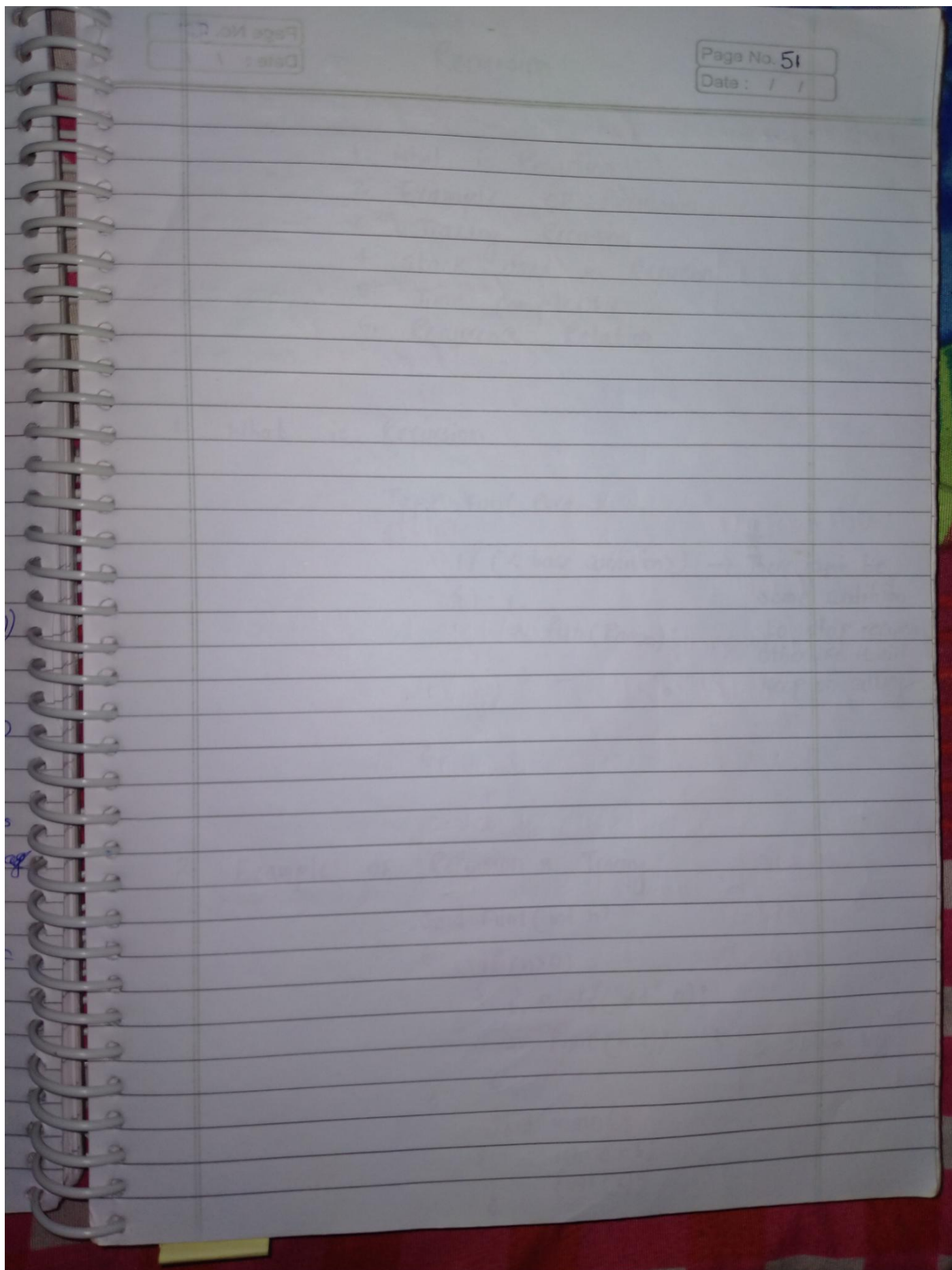
Note:- Why we express the running time of a given algorithm as a function of the input size n (i.e. $f(n)$)

Reason:-

Execution Times? Not a good measure as execution time are specific to a particular computer

Number of statements executed? Not a good measure as a no. of statement varies with programming language as well as the style of individual programmer

Ideal solution? → Let us express it as function of input size n & comparison is independent of machine time, programming style etc.



Page No. 52
Date : / /

Page No. 52
Date : / /